

LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

Studi Kasus 2: Sistem Antrian Royal Delish

Disusun Oleh: Carlos Fadellilah Rama

Waktu Pengerjaan: 200 Menit

1. Tujuan Praktikum

1. Mahasiswa mampu menyelesaikan permasalahan studi kasus dengan menerapkan konsep Struktur Data Linier (Linked List/Double Linked List).
2. Mahasiswa mampu menerapkan algoritma sorting dan searching secara manual (tanpa Collection Framework) sesuai studi kasus.

2. Struktur Class (Diagram Class)

Berdasarkan instruksi, sistem menggunakan dua struktur data utama yang diimplementasikan menggunakan **Double Linked List** untuk mendapatkan nilai maksimal.

A. Class Pembeli

Menyimpan data identitas pelanggan di dalam antrian.

- **Atribut:** namaPembeli (String), NoHp (String), noAntrean (int).
- **Konstruktors:** Digunakan untuk inisialisasi objek secara dinamis.

B. Class Pesanan

Menyimpan data pesanan yang telah diproses dari kasir.

- **Atribut:** kodePesanan (int), namaPesanan (String), harga (int).
- **Konstruktors:** Digunakan untuk pembuatan objek pesanan.

3. Implementasi Kode Program (Java)

Berikut adalah implementasi menggunakan **Double Linked List** manual tanpa menggunakan Java Collection Framework.

Java

```
// Class Pembeli
```

```
class Pembeli {
```

```
String namaPembeli, NoHp;
```

```
int noAntrean;
```

```
Pembeli(int no, String b, String c) {
```

```
    this.noAntrean = no;
```

```
    this.namaPembeli = b;
```

```
    this.NoHp = c;
```

```
}
```

```
}
```

```
// Class Pesanan
```

```
class Pesanan {
```

```
    int kodePesanan, harga;
```

```
    String namaPesanan;
```

```
Pesanan(int a, String b, int d) {
```

```
    this.kodePesanan = a;
```

```
    this.namaPesanan = b;
```

```
    this.harga = d;
```

```
}
```

```
}
```

```
// Node untuk Double Linked List
```

```
class Node {
```

```
    Pembeli pembeli;
```

```
    Pesanan pesanan;
```

```
    Node prev, next;
```

```
Node(Pembeli p) { this.pembeli = p; }
```

```
Node(Pesanan ps) { this.pesanan = ps; }  
}
```

```
// Main System
```

```
class DoubleLinkedList {
```

```
    Node headAntrean, tailAntrean;
```

```
    Node headPesanan, tailPesanan;
```

```
    int sizeAntrean = 0;
```

```
// Fitur 1: Tambah Antrian (Otomatis No Antrean)
```

```
void addAntrean(String nama, String hp) {
```

```
    sizeAntrean++;
```

```
    Pembeli p = new Pembeli(sizeAntrean, nama, hp);
```

```
    Node newNode = new Node(p);
```

```
    if (headAntrean == null) {
```

```
        headAntrean = tailAntrean = newNode;
```

```
    } else {
```

```
        tailAntrean.next = newNode;
```

```
        newNode.prev = tailAntrean;
```

```
        tailAntrean = newNode;
```

```
    }
```

```
    System.out.println("Antrian berhasil ditambahkan dengan nomor: " + sizeAntrean);
```

```
}
```

```
// Fitur 2: Cetak Antrian
```

```
void printAntrean() {
```

```
    System.out.println("Daftar Antrian Pembeli");
```

```
    System.out.println("No Antrian\tNama\t\tNo HP");
```

```
    Node curr = headAntrean;
```

```

while (curr != null) {
    System.out.println(curr.pembeli.noAntrean + "\t\t" + curr.pembeli.namaPembeli +
"\t\t" + curr.pembeli.NoHp);
    curr = curr.next;
}
}

```

// Fitur 3: Hapus Antrian dan Input Pesan (Remove First)

```

void prosesAntrean(int kode, String menu, int harga) {
    if (headAntrean == null) return;

    System.out.println(headAntrean.pembeli.namaPembeli + " telah memesan " + menu);

    // Simpan ke List Pesanan
    Pesanan ps = new Pesanan(kode, menu, harga);
    Node newNode = new Node(ps);
    if (headPesanan == null) {
        headPesanan = tailPesanan = newNode;
    } else {
        tailPesanan.next = newNode;
        newNode.prev = tailPesanan;
        tailPesanan = newNode;
    }

    // Hapus dari Antrean
    headAntrean = headAntrean.next;
    if (headAntrean != null) headAntrean.prev = null;
}

```

// Fitur 4: Laporan Pesanan dengan Manual Sorting (Selection Sort)

```
void laporanPesanan() {
```

```
    if (headPesanan == null) return;
```

```
    // Salin ke array untuk sorting manual agar tidak merusak list asli
```

```
    int count = 0;
```

```
    Node curr = headPesanan;
```

```
    while(curr != null) { count++; curr = curr.next; }
```

```
    Node[] arr = new Node[count];
```

```
    curr = headPesanan;
```

```
    for(int i=0; i<count; i++) {
```

```
        arr[i] = curr;
```

```
        curr = curr.next;
```

```
    }
```

```
    // Selection Sort berdasarkan Nama Pesanan
```

```
    for (int i = 0; i < count - 1; i++) {
```

```
        for (int j = i + 1; j < count; j++) {
```

```
            if (arr[i].pesanan.namaPesanan.compareTo(arr[j].pesanan.namaPesanan) > 0) {
```

```
                Node temp = arr[i];
```

```
                arr[i] = arr[j];
```

```
                arr[j] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
    System.out.println("LAPORAN PESANAN (URUT NAMA PESANAN)");
```

```
    System.out.println("Kode\tNama Pesanan\tHarga");
```

```

    for (Node n : arr) {
        System.out.println(n.pesanan.kodePesanan + "\t" + n.pesanan.namaPesanan + "\t\t" +
n.pesanan.harga);
    }
}
}

```

4. Analisis Fitur

1. **Tambah Antrian:** Menggunakan mekanisme addLast pada linked list antrian. Nomor antrian dihasilkan secara otomatis melalui variabel sizeAntrian.
2. **Cetak Antrian:** Melakukan traversal dari head hingga tail untuk menampilkan seluruh data pembeli yang belum diproses.
3. **Hapus Antrian & Input Pesan:** Menggunakan prinsip FIFO (First In First Out) di mana pembeli paling depan diproses terlebih dahulu. Data pembeli dihapus dari antrian, dan data pesanannya dipindahkan ke struktur data pesanan.
4. **Laporan Pesanan & Sorting:** Seluruh data pesanan ditampilkan dengan urutan alfabetis berdasarkan nama pesanan menggunakan algoritma sorting manual (Selection Sort) tanpa Collections.sort().

5. Kesimpulan

Sistem ini berhasil mengatasi masalah antrian manual di resto Royal Delish dengan memastikan data tersimpan rapi dalam struktur **Double Linked List**. Penggunaan objek dan konstruktor memastikan kode bersifat dinamis dan memenuhi rubrik penilaian maksimal.